Sistema de Gestao

Especificacao de Sistema (SSD v1.5 — Final)

Versao 1.5 — Abril 2026 — Para uso no Claude Code

1. Visao Geral

Centralizar o controle financeiro e operacional de vendas e servicos, cobrindo o ciclo completo: cadastro → orcamento → aprovacao/pos-venda → fluxo de caixa.

Target Users

Internal sales and management team.

MVP Scope

- Client, product, and kit management
- Quotes with multi-version support
- Approval and payment tracking
- Cash flow management
- Fixed expenses management
- PDF report exports
- Inventory control linked to products
- Dashboard with KPIs
- Authentication with login, multiple users and role-based access control (RBAC)

2. Security

> *Security is a first-class concern throughout the entire stack. All guidelines below are mandatory, not optional.*

2.1. Credentials & Secrets

- All secrets (DB_URL, JWT_SECRET, etc.) stored exclusively in environment variables
- Never commit .env files — .env.example with placeholder values only
- Secrets injected at runtime via the deployment platform (Railway environment variables panel)
- Rotate JWT_SECRET and DB credentials immediately if accidentally exposed
- Use strong randomly-generated values for all secrets (minimum 32 bytes entropy)

2.2. Authentication & Authorization

- JWT-based authentication (access token + refresh token pattern)
- Access tokens: short-lived (15 minutes)
- Refresh tokens: longer-lived (7 days), stored as HttpOnly cookies (never localStorage)
- Passwords hashed with bcrypt (minimum 12 salt rounds)
- RBAC enforced on every API endpoint via middleware — never trust client-side role checks alone
- Failed login attempts rate-limited (max 5 attempts per 15 min per IP)
- Session invalidated on logout by blacklisting the refresh token

2.3. API Security

- All endpoints protected by HTTPS in production (Railway provides TLS by default)
- CORS configured to allow only the frontend origin — no wildcard (*)
- All user inputs validated and sanitized server-side with Zod before reaching business logic
- Parameterized queries or ORM only — never raw string concatenation in SQL
- HTTP security headers enforced via Helmet.js (X-Frame-Options, HSTS, CSP, etc.)
- File uploads (if applicable) validated for type, size, and scanned for malicious content

2.4. Frontend Security

- No secrets, API keys, or internal IDs exposed in client-side code or localStorage
- All API calls include Authorization header with Bearer token
- Sensitive routes protected by auth guards — redirect to login if token is invalid/expired
- XSS prevention: never use dangerouslySetInnerHTML; always sanitize user-generated content

2.5. Role-Based Access Control (RBAC)

Role	Permissions
Administrator	Full access: all modules, user management, reports, settings
Sales	Clients, products, kits, quotes. No access to cash flow, reports, or user management
Finance	Cash flow, fixed expenses, installments, reports. No quote editing

> *Backend must re-validate the user role on every protected request. Frontend role checks are UX only.*

3. Client Management

Features: full CRUD (create, list, search, edit, delete).

Fields

Field	Required	Notes
name / companyName	✅	
taxId (CPF/CNPJ)	✅	Format-validated; unique per client
nationalId (RG)	—	Optional
address	—	Optional
zipCode	—	Auto-fills address via ViaCEP API
email	—	Optional
phone (WhatsApp)	—	Used for quote delivery via wa.me link

Business Rules

- taxId must be unique across all clients
- A client can have multiple quotes
- Deletion blocked if the client has linked quotes

4. Product Management

Features: full CRUD.

Fields

Field	Required	Notes
name	✅	
basePrice	✅	Cost / purchase price
markupPercent	✅	
finalPrice	—	Calculated: basePrice * (1 + markupPercent / 100)
unit	—	e.g. unit, m2, m, hour
stockQty	✅	Current quantity in stock
minStock	—	Alert threshold
isActive	✅	Soft-delete flag

Business Rules

- finalPrice recalculated automatically on basePrice or markupPercent change
- Inactive products excluded from new quote item selectors
- Quote approval auto-deducts stockQty for each sold item
- Dashboard alert shown when stockQty < minStock

4.1. Kit Management

A kit is a pre-configured bundle of products for faster quote creation. Features: full CRUD.

Fields

Field	Required	Notes
name	✅	
items (product + qty)	✅	References to existing products
totalPrice	—	Calculated: sum of (finalPrice * qty) for each item

Business Rules

- A kit can be added to a quote as a single unit or expanded into individual items
- Product price changes propagate to kit totalPrice automatically

5. Quote Management

Features: create, list, edit, version, send.

Fields

Field	Required	Notes
clientId	✅	
items (product/kit + qty + unitPrice)	✅	
subtotal	—	Calculated
laborCost	—	Manual input, separate from products
discount	—	Fixed value or percentage
total	—	subtotal + laborCost - discount
status	✅	pending_review | accepted | rejected | no_response
version	—	Auto-incremented per quote
createdAt	—	Auto
updatedAt	—	Auto

Status Flow

```
pending_review  →  accepted
                →  rejected
                →  no_response
```

WhatsApp Delivery

The system generates a wa.me link with a pre-formatted message containing the quote summary (client, items, total, validity). The link opens WhatsApp Web or the mobile app with the message ready to send. No paid API required.

Business Rules

- Each edit creates a new version; older versions are read-only
- The latest version is the active quote
- Quotes with status pending_review for more than 5 days appear in a dedicated stale-quotes list with a follow-up suggestion

6. Approval & Post-Sale

Status transition: accepted quote → generates a sale record.

Payment Configuration

Field	Required	Notes
paymentType	✅	lump_sum | installment
downPayment	—	Fixed value or % of total
installmentCount	✅ (if installment)	
dueDates	✅	One per installment
remainingAmount	—	Calculated: total - downPayment
installmentValue	—	Calculated: remainingAmount / installmentCount

Business Rules

- downPayment reduces the installable amount automatically
- Each installment creates a future cash flow entry with its due date
- Installment status: pending | paid | overdue
- Quote approval auto-deducts product stock
- Cancelling an approved quote reverses the stock deduction

7. Fixed Expenses

Dedicated screen for managing recurring expenses (rent, salaries, subscriptions, etc.).

Fields

Field	Required	Notes
name / description	✅	e.g. Rent, Internet, Accountant
amount	✅	
dueDay	✅	Day of month (1-31)
category	—	e.g. Infrastructure, Personnel, Services
isActive	✅	Pause without deleting

Business Rules

- Active fixed expenses appear on the dashboard as projected outflows for the current month
- Each month, the system auto-generates entries in the cash flow
- User marks the monthly entry as paid, recording the actual outflow
- Payment history available per fixed expense

8. Cash Flow

Transaction Fields

Field	Required	Notes
type	✅	income | expense
amount	✅	
date	✅	
origin	✅	manual | approved_quote | installment | fixed_expense
description	—	Optional
referenceId	—	ID of linked quote / installment / fixed expense

Views

- Current balance (income - expenses up to today)
- Total receivable (future pending installments)
- Paginated history with period and type filters
- Monthly summary: income vs. expenses chart

Business Rules

- Lump-sum approval creates an immediate income entry
- Installment approval creates future income entries, confirmed when marked as paid
- Fixed expenses generate projected monthly outflow entries automatically

9. Inventory Control

Inventory is managed directly on products (stockQty, minStock). Movements are automatic or manual.

Movement Types

Type	Description
Manual inflow	Stock replenishment entered by user
Automatic outflow	Deducted on quote approval
Manual outflow	Loss, adjustment, or disposal
Inventory adjustment	Direct correction of current quantity

Business Rules

- Quote approval automatically deducts items from stock
- Cancelling an approved quote reverses the stock deduction
- Dashboard shows an alert for products below minStock
- Full movement history available per product

10. Reports (PDF Export)

Exportable PDF reports. Accessible to Administrator and Finance roles only.

Report	Content
Sales Report	Approved quotes by period, with totals and clients
Cash Flow	Income and expenses by period, with final balance
Accounts Receivable	Pending installments with due dates
Fixed Expenses	Monthly fixed expense list and payment status
Inventory	Current stock levels with low-stock alerts
Stale Quotes	Quotes stuck in pending_review with elapsed time

Business Rules

- PDF generated server-side (Puppeteer recommended for layout fidelity)
- All reports include a header with company name, generated date, and applied period filter
- Period filters available on all reports

11. Dashboard

Indicator	Description
Total cash balance	Current balance
Total receivable	Sum of pending installments
Approved quotes	Count for current month
Pending quotes	pending_review + no_response

Indicator	Description
Stale quotes	In pending_review for more than 5 days
Fixed expenses (month)	Projected total vs. paid total
Monthly revenue	Income entries for current month
Low stock products	Products below minStock

12. Data Relationships

```
User                >── Role
Client              1 ──< Quotes
Quote               1 ──< QuoteItems    >── Product
Quote               1 ──< QuoteItems    >── Kit
Kit                 1 ──< KitItems       >── Product
Quote               1 ──< QuoteVersions
AcceptedQuote       1 ──< Installments
Installment         1 ──  CashTransaction
FixedExpense        1 ──< FixedExpenseLogs ── CashTransaction
Product             1 ──< StockMovements
```

13. Business Rules (Consolidated)

1. taxId must be unique per client
2. Inactive products excluded from new quotes
3. Quotes are versioned; older versions are read-only
4. laborCost is manual input, not linked to any product
5. Discounts can be a fixed value or a percentage
6. Lump-sum approval → immediate cash inflow + stock deduction
7. Installment approval → future cash entries, confirmed on payment
8. downPayment reduces the installable amount automatically
9. Stale-quote alert fires automatically after 5 days in pending_review
10. Client deletion blocked if linked quotes exist
11. Fixed expenses auto-generate monthly cash outflow entries
12. Cancelling an approved quote reverses the stock deduction
13. PDF reports restricted to Administrator and Finance roles
14. All RBAC rules enforced server-side on every request

14. Implementation

14.1. Tech Stack

Layer	Choice
Build tooling	Vite 8 (Rolldown bundler)
Framework	React 19+ (use, Actions, Transitions, Suspense)
Language	TypeScript — strict mode enabled
Routing	TanStack Router (file-based, type-safe, search param validation)
Server state	TanStack Query v5+
Client state	Zustand (minimal, for UI-only global state)
Styling	SCSS + CSS Modules
Schema validation	Zod (API responses, forms, env vars)
Path aliases	@/ prefix configured in vite.config.ts
HTTP client	Axios or fetch wrapper (no credentials in source)
Backend runtime	Node.js (Express)
Database	PostgreSQL
ORM / Query	Prisma (type-safe, migration versioning)
Auth	JWT (access + refresh token) + bcrypt (>=12 rounds)
Security headers	Helmet.js
Input validation	Zod (server-side, on every endpoint)
PDF generation	Puppeteer (server-side rendering)
WhatsApp delivery	wa.me link — no paid API
Deploy	Railway (Node + Postgres, auto-deploy from GitHub)

14.2. Frontend Architecture

> *All source code, variable names, comments, and file names must be in English.*

Folder Structure

```
src/
├── assets/          # Static files (images, icons)
├── components/      # Global shared UI components (UI Kit)
├── config/          # Environment variables & global constants
├── features/        # Business logic organized by domain
│   └── quotes/
│       ├── api/         # Feature-specific API calls
│       ├── components/  # Components used only in this feature
│       ├── hooks/       # Feature-specific logic
│       ├── types/       # TypeScript definitions for this feature
│       └── index.ts     # Public API (barrel file)
├── hooks/           # Global reusable hooks
├── layouts/         # Shared page layouts (Nav, Sidebar)
├── pages/           # Route entry points (TanStack Router)
├── providers/       # Global context providers (Theme, Auth, Query)
├── stores/          # Global state (Zustand)
```

```
├── utils/           # Shared utility functions
└── main.tsx         # App entry point
```

Core Architectural Pillars

- Feature isolation: other parts of the app import only from a feature's index.ts (barrel file) — never from internal paths
- Type safety: TypeScript strict mode + Zod for runtime validation of API responses and form data
- Server state: all remote data managed exclusively by TanStack Query — no manual fetch/useEffect for data fetching
- Client state: Zustand reserved for UI-only state (modals, sidebar, theme) — keep it minimal
- Routing: TanStack Router with file-based routes and type-safe search params
- Path aliases: use @/components/Button, never ../../../components/Button
- Streaming UIs: use React Suspense + streaming for incremental loading patterns

14.3. Backend Architecture

Folder Structure

```
src/
├── controllers/     # Route handlers
├── services/        # Business logic
├── repositories/    # Database queries (Prisma)
├── models/          # Prisma schema & types
├── middlewares/     # Auth (JWT), RBAC, validation, error handler
├── routes/          # REST route definitions
├── utils/           # Helpers, formatters
└── prisma/          # schema.prisma + migrations
```

Conventions

- REST pattern for all endpoints
- Zod validation on every request body and query param before reaching business logic
- Prisma for all database access — no raw string SQL concatenation ever
- Migrations versioned in prisma/migrations/ and committed to the repository
- Environment variables validated at startup via Zod (fail fast on missing secrets)
- Centralized error handler middleware — no try/catch leaking HTTP logic into services

14.4. Deployment (Railway)

Item	Detail
Platform	Railway
Services	Node.js API + PostgreSQL (both in the same Railway project)
Auto-deploy	Push to main branch triggers deploy automatically
Secrets	Defined in Railway environment variables panel — never in code
Free tier	$5 credit/month — suitable for development and staging
Production plan	~$5-20/month depending on usage
TLS	Provided by Railway by default (HTTPS enforced)
DB backups	Enable automatic backups in Railway Postgres settings

> *Never commit .env files. Use .env.example with placeholder values as the only committed reference. Rotate all credentials immediately if a secret is ever exposed.*

15. Offline Resilience

> *Strategy: Option A — Local Draft Auto-save. Covers data loss during short connection drops (seconds to minutes). Full offline PWA support is deferred to a future phase.*

15.1. Behaviors

Network Status Detection

A global useOnlineStatus hook monitors navigator.onLine and the browser online/offline events. The result is exposed via a provider so any component can react to connectivity changes.

```
// hooks/useOnlineStatus.ts
export function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);
  useEffect(() => {
    window.addEventListener('online',  () => setIsOnline(true));
    window.addEventListener('offline', () => setIsOnline(false));
  }, []);
  return isOnline;
}
```

Auto-save Draft (Quote Form)

While the user fills in the quote form, the current state is persisted to IndexedDB on every change via a useQuoteDraft hook. This protects against data loss during connection drops, page refreshes, or accidental navigation.

```
// features/quotes/hooks/useQuoteDraft.ts
const DRAFT_KEY = 'quote_draft';
export function useQuoteDraft() {
  const saveDraft = (data: QuoteDraft) => idb.set(DRAFT_KEY, data);
  const loadDraft = () => idb.get<QuoteDraft>(DRAFT_KEY);
  const clearDraft = () => idb.del(DRAFT_KEY);
  return { saveDraft, loadDraft, clearDraft };
}
```

UI Feedback

- Offline banner: a persistent top bar appears when the connection is lost — "You are offline. Your draft is saved locally."
- Submit button disabled while offline — tooltip: "Reconnect to save your quote."
- Draft recovery prompt: when the user reopens the quote form and a saved draft exists, a banner offers to restore it — "You have an unsaved draft. Restore it?"
- Online restoration banner: when the connection returns — "You're back online." — pending mutations are retried automatically.

Automatic Retry on Reconnect

TanStack Query handles retry and reconnect natively. Configure globally in the QueryClient:

```
// providers/QueryProvider.tsx
new QueryClient({
  defaultOptions: {
    queries: { refetchOnReconnect: true, retry: 2 },
    mutations: { retry: 1 },
  },
})
```

15.2. Libraries

- idb-keyval — lightweight IndexedDB wrapper for draft persistence (get / set / del)
- TanStack Query v5 — handles refetchOnReconnect and mutation retry natively
- React 19 Suspense + Transitions — for non-blocking UI updates during reconnection

15.3. Scope & Limitations

- Draft auto-save applies to the quote creation and editing form only — not to all modules
- Browsing existing quotes, clients, and products while offline is not supported in MVP — requires PWA (Phase 2)
- Draft is stored per browser — not synced between devices or users
- Draft is cleared automatically after a successful quote submission

16. UI / UX Specification

> *All UI decisions are final for MVP. Claude Code must implement exactly as specified — do not improvise layout, colors, or interaction patterns.*

16.1. Design Principles

- Mobile-first: design for a 390px screen first, then scale up to desktop
- Zero onboarding: every screen must be self-explanatory — no tooltips required to operate
- Progressive disclosure: show only what is needed for the current task; advanced options are secondary
- Elegant restraint: avoid visual noise — no gradients, no shadows stacked, no more than 2 accent colors per screen
- Forgiving inputs: auto-format CPF/CNPJ/phone as the user types; never block submission with cryptic errors
- Destructive actions always require a confirmation modal — never a single click

16.2. Color Palette (Design Tokens)

Use CSS custom properties defined globally in styles/tokens.scss. Never hardcode hex values in components.

Base Palette

--color-primary: #2C5282 — Main interactive elements, nav, active states

--color-primary-light: #4A90D9 — Hover states, links

--color-primary-bg: #EBF4FF — Subtle primary backgrounds, selected rows

--color-success: #276749 — Approved status, positive values

--color-success-bg: #F0FAF4 — Success backgrounds

--color-warning: #B7791F — Pending / stale quotes alert

--color-warning-bg: #FFFBEA — Warning backgrounds

--color-danger: #C53030 — Rejected status, delete actions

--color-danger-bg: #FFF0F0 — Danger backgrounds

--color-neutral-900: #1A202C — Primary text

--color-neutral-600: #4A5568 — Secondary text, labels

--color-neutral-300: #CBD5E0 — Borders, dividers

--color-neutral-100: #F7FAFC — Page background, table alternating rows

--color-surface: #FFFFFF — Cards, modals, form backgrounds

> *Rationale: the palette uses deep blue-slate as primary (professional, calm, non-fatiguing) with low-saturation status colors. Avoid pure black (#000000) and pure white backgrounds — use neutral-900 and neutral-100 instead.*

16.3. Typography

Font family: Inter (Google Fonts) — fallback: system-ui, sans-serif

Base size: 16px (1rem) — never go below 14px for any readable text

Scale: 12 / 14 / 16 / 18 / 20 / 24 / 30 / 36px

Page title: 24-30px, weight 700

Section heading: 18-20px, weight 600

Body: 16px, weight 400

Label / caption: 14px, weight 500, color neutral-600

Micro / badge: 12px, weight 600, uppercase, letter-spacing 0.05em

16.4. Spacing & Layout

Use an 8px base grid. All spacing values must be multiples of 8 (8, 16, 24, 32, 40, 48, 64px).

Breakpoints

sm (< 640px): Mobile (primary design target)

md (640-1024px): Tablet

lg (> 1024px): Desktop

Layout Grid

- Mobile: single column, full-width content, bottom navigation bar
- Tablet/Desktop: fixed sidebar (240px) + main content area
- Max content width: 1200px, centered
- Page padding: 16px mobile, 32px desktop
- Card padding: 16px mobile, 24px desktop

16.5. Navigation

Mobile — Bottom Tab Bar

5 tabs maximum. Icon + short label. Active tab uses --color-primary. Inactive uses --color-neutral-600.

- Dashboard — LayoutDashboard icon
- Quotes — FileText icon
- Clients — Users icon
- Products — Package icon
- More — Menu icon — opens a drawer with: Fixed Expenses, Reports, Cash Flow, Settings

Desktop — Left Sidebar

Full labels, grouped by category. Collapsible to icon-only on smaller desktops. Sticky, does not scroll with content.

16.6. Component Patterns

Buttons

- Primary: bg primary, white text — main CTA per screen (only 1 per view)
- Secondary: border primary, primary text — secondary actions
- Ghost: no border, primary text — tertiary actions (e.g. Cancel)
- Danger: bg danger — destructive actions, always behind a confirmation modal
- Icon: 40x40px tap target minimum — never smaller on mobile

Forms

- Input height: 44px minimum (touch target)
- Label always above the input — never placeholder-only labels
- Inline validation: show error below the field on blur, not on submit

- Auto-format masks: CPF (000.000.000-00), CNPJ (00.000.000/0000-00), phone ((00) 00000-0000), currency (R$ 0.000,00)
- Required fields marked with a red asterisk — explained once at top of form
- Submit button disabled while form is invalid or while request is in-flight

Tables & Lists

- Desktop: full table with sortable column headers
- Mobile: card list — each row becomes a card with primary info visible and secondary info collapsed
- Empty state: illustration + short message + primary CTA (e.g. 'No quotes yet — Create your first quote')
- Loading state: skeleton rows — never a full-page spinner
- Alternating row background: --color-neutral-100 / --color-surface

Status Badges

Pill shape, 12px uppercase text, 6px horizontal padding. Map to semantic colors:

- pending_review → warning-bg + warning text
- accepted → success-bg + success text
- rejected → danger-bg + danger text
- no_response → neutral-100 + neutral-600 text
- overdue → danger-bg + danger text
- paid → success-bg + success text

Modals

- Mobile: bottom sheet (slides up from bottom, 90% screen height max)
- Desktop: centered modal, max-width 560px, backdrop blur
- Always include a visible close button (X) in the top-right corner
- Confirmation modal for destructive actions: title + consequence description + Cancel (ghost) + Confirm (danger)

Toast Notifications

- Position: top-right desktop, top-center mobile
- Duration: 4 seconds auto-dismiss; errors stay until manually dismissed
- Types: success (green), error (red), warning (yellow), info (blue)
- Never use browser alert() — always use the toast system

16.7. Key Screen Flows

Quote Creation Flow

- Client selector (searchable dropdown) → if not found, inline 'Add new client' shortcut
- Item builder: search product/kit, set quantity, see unit price and line total — running total updates live at bottom
- Labor cost field (optional, numeric, currency formatted)
- Discount field (toggle between % and fixed value)
- Grand total prominent at bottom — large, bold, always visible
- WhatsApp send button: generates wa.me link, copies to clipboard with one tap
- Save as draft (local) when offline — banner confirms draft saved

Dashboard Layout

- Top row: 2-column grid of KPI cards on mobile, 4-column on desktop
- Stale quotes alert: red-bordered card with count — tapping opens the stale quotes list
- Low stock alert: orange-bordered card — tapping opens inventory
- Monthly chart: income vs. expenses bar chart, current month highlighted

16.8. Accessibility (Minimum Requirements)

- Color contrast: minimum 4.5:1 for body text, 3:1 for large text (WCAG AA)
- All interactive elements keyboard-navigable
- All images and icons have aria-label or aria-hidden as appropriate
- Form inputs always have associated <label> elements
- Focus ring never removed — customize with --color-primary outline

17. Claude Code — Implementation Instructions

> *This section is written directly for Claude Code. Read it entirely before writing any code. These are constraints, not suggestions.*

17.1. Project Bootstrap

Run these commands exactly to initialize the project:

```
# Frontend
npm create vite@latest client -- --template react-ts
cd client && npm install
npm install @tanstack/react-query @tanstack/react-router
npm install zustand zod idb-keyval axios
npm install -D sass @types/node
```

```
# Backend
mkdir server && cd server && npm init -y
npm install express prisma @prisma/client zod bcryptjs jsonwebtoken helmet cors dotenv
npm install -D typescript @types/express @types/node @types/bcryptjs
@types/jsonwebtoken ts-node nodemon
npx prisma init
```

17.2. Non-Negotiable Rules

- All code, variable names, comments, file names and commit messages must be in English
- TypeScript strict mode must be enabled in tsconfig.json — never use 'any'
- Never store secrets in source code — read all config from process.env
- Validate process.env at startup with Zod — crash immediately if required vars are missing
- Never use raw SQL string concatenation — use Prisma exclusively for database access
- Never trust client-supplied role or userId — always read from the decoded JWT on the server
- Every API route must go through the auth middleware unless explicitly marked as public
- Never use dangerouslySetInnerHTML
- Never import from inside another feature's folder — only from its index.ts barrel file
- Never use localStorage for tokens — use HttpOnly cookies for refresh token, memory for access token

17.3. Environment Variables

Required variables — validated at startup via Zod in server/src/config/env.ts:

```
DATABASE_URL=postgresql://user:password@host:5432/dbname
JWT_ACCESS_SECRET=<min 32 chars random string>
JWT_REFRESH_SECRET=<min 32 chars random string>
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=https://your-frontend-domain.com
NODE_ENV=production
```

Frontend (Vite) — prefix all with VITE_ so they are injected at build time:

```
VITE_API_BASE_URL=https://your-api-domain.com
```

17.4. Folder Scaffold (generate this structure first)

Frontend — src/:

```
assets/          components/      config/env.ts
features/
  clients/        products/        kits/
  quotes/         approvals/       cash-flow/
  fixed-expenses/ inventory/       reports/
  auth/           dashboard/
hooks/           layouts/          pages/
providers/       stores/           utils/
styles/tokens.scss               main.tsx
```

Backend — src/:

```
config/env.ts    controllers/     services/
repositories/    middlewares/     routes/
utils/           prisma/schema.prisma
```

17.5. Implementation Order

> *Follow this order strictly. Do not start a phase before the previous one is working and tested.*

- Phase 1 — Foundation: env validation, Prisma schema (all models), database migrations, Express server skeleton with Helmet + CORS
- Phase 2 — Auth: register, login, refresh token, logout endpoints; JWT middleware; RBAC middleware
- Phase 3 — Core CRUD: clients, products, kits (API + frontend pages)
- Phase 4 — Quotes: creation with versioning, status flow, WhatsApp link generation
- Phase 5 — Approval & Payments: installment generation, cash flow entries
- Phase 6 — Cash Flow & Fixed Expenses: transaction listing, fixed expense auto-generation
- Phase 7 — Inventory: stock movement on approval/cancellation, low-stock alerts
- Phase 8 — Dashboard: KPI aggregations, stale quote detection
- Phase 9 — Reports: PDF generation with Puppeteer
- Phase 10 — Offline Resilience: useOnlineStatus, useQuoteDraft with idb-keyval, QueryClient retry config
- Phase 11 — Polish: loading skeletons, empty states, toast notifications, mobile nav

17.6. Prisma Schema — Entity List

Generate a schema.prisma with the following models and relationships:

- User: id, email, passwordHash, role (ADMIN|SALES|FINANCE), isActive, createdAt
- Client: id, name, taxId (unique), nationalId?, address?, zipCode?, email?, phone?, createdAt
- Product: id, name, basePrice, markupPercent, finalPrice, unit?, stockQty, minStock?, isActive
- Kit: id, name, totalPrice (computed), items: KitItem[]
- KitItem: id, kitId, productId, quantity
- Quote: id, clientId, status, activeVersionId?, createdAt, updatedAt
- QuoteVersion: id, quoteId, version, subtotal, laborCost, discount, total, createdAt
- QuoteItem: id, quoteVersionId, productId?, kitId?, quantity, unitPrice, lineTotal
- Sale: id, quoteId, paymentType, downPayment, total, createdAt
- Installment: id, saleId, dueDate, amount, status (PENDING|PAID|OVERDUE), paidAt?
- CashTransaction: id, type (INCOME|EXPENSE), amount, date, origin, description?, referenceId?
- FixedExpense: id, name, amount, dueDay, category?, isActive
- FixedExpenseLog: id, fixedExpenseId, month, year, status (PENDING|PAID), paidAt?
- StockMovement: id, productId, type (INFLOW|OUTFLOW|ADJUSTMENT), quantity, reason, createdAt

17.7. API Route Reference

Implement REST endpoints for each resource. Use this naming convention:

- POST /auth/login — Public
- POST /auth/refresh — Public
- POST /auth/logout — Authenticated
- GET /clients — SALES | ADMIN
- POST /clients — SALES | ADMIN
- GET /clients/:id — SALES | ADMIN
- PUT /clients/:id — SALES | ADMIN
- DELETE /clients/:id — ADMIN
- GET /products — SALES | ADMIN
- POST /products — ADMIN
- PUT /products/:id — ADMIN
- DELETE /products/:id — ADMIN
- GET /kits POST /kits PUT /kits/:id — ADMIN
- GET /quotes — SALES | ADMIN
- POST /quotes — SALES | ADMIN
- GET /quotes/:id/versions — SALES | ADMIN
- POST /quotes/:id/accept — SALES | ADMIN
- POST /quotes/:id/reject — SALES | ADMIN
- GET /quotes/stale — SALES | ADMIN
- POST /sales/:id/installments — FINANCE | ADMIN
- PATCH /installments/:id/pay — FINANCE | ADMIN
- GET /cash-flow — FINANCE | ADMIN
- POST /cash-flow — FINANCE | ADMIN
- GET /fixed-expenses — FINANCE | ADMIN
- POST /fixed-expenses — FINANCE | ADMIN
- PATCH /fixed-expenses/:id/pay/:month — FINANCE | ADMIN
- GET /inventory/movements — ADMIN
- POST /inventory/movements — ADMIN
- GET /dashboard — ALL roles
- GET /reports/:type — FINANCE | ADMIN — streams PDF

17.8. Design System Implementation

Start by creating styles/tokens.scss with all CSS custom properties from Section 16.2. Then create these global components before building any feature:

- components/Badge/Badge.tsx — status badges per Section 16.6
- components/Button/Button.tsx — all 5 variants per Section 16.6
- components/Input/Input.tsx — with label, error, mask support
- components/Modal/Modal.tsx — bottom sheet on mobile, centered on desktop
- components/Toast/Toast.tsx + providers/ToastProvider.tsx
- components/Skeleton/Skeleton.tsx — for loading states
- components/EmptyState/EmptyState.tsx — icon + message + optional CTA
- layouts/AppLayout.tsx — bottom tab bar (mobile) + sidebar (desktop)

> *Build the design system components first. Every feature screen is assembled from these primitives. Consistency depends on this foundation.*

17.9. Key Technical Decisions Already Made

Do not reconsider these — they are final:

- Currency: always store as integers (cents) in the database — display formatted with Intl.NumberFormat('pt-BR')
- Dates: store as UTC ISO strings in DB — display in America/Sao_Paulo timezone using date-fns-tz
- Pagination: cursor-based for all list endpoints (not offset) — default page size 20
- Soft deletes: use isActive flag — never hard-delete clients, products or kits
- Quote totals: always recomputed on the server before saving — never trust client-sent totals
- PDF generation: Puppeteer renders an HTML template server-side — do not use client-side PDF libs
- WhatsApp: build wa.me URL as: https://wa.me/55{phone}?text={encodeURIComponent(message)}
- Error responses: always return { error: string, code: string } — never expose stack traces in production